Đề 06 · Nhập môn Công nghệ nền tảng (RIPT1304) · Tài liệu học nội bộ, không nộp

Hiểu và làm chủ phần việc của Lê Đình Thành

Sinh viên 2 · Frontend Developer, UI/UX, Media/CDN Engineer, Frontend QA & Web Performance, biên tập hồ sơ và video demo

Bạn giữ phần sinh viên nhìn thấy và dùng: 3 học liệu, trang /hoc-lieu, đường đi của video từ máy chủ tới trình duyệt, và các số đo phía trình duyệt. Phần quan trọng nhất về mặt thiết kế hệ thống là: video đi đường nào, tốn bao nhiêu băng thông, ai được xem.

Mục lục

1. Phần 1. Kiến thức nền cả nhóm phải nắm
2. Phần 2. Phần việc của bạn
3. Phần 3. Học liệu: video, HLS, mã băm
4. Phần 4. Video đi đường nào: CDN, Nginx, liên kết có hạn
5. Phần 5. Trang web học liệu
6. Phần 6. Đo phía trình duyệt và Kiểm thử 1
7. Phần 7. Sửa và cấu hình trực tiếp
8. Phần 8. Chạy lại và số liệu của bạn
9. Phần 9. Câu hỏi vấn đáp
0. Phần 10. Việc cần làm trước khi nộp và bảo vệ

> Cách đọc: đọc phần 1 trước (kiến thức nền chung, ai cũng phải nắm), rồi đọc lần lượt các phần riêng của bạn. Chỗ nào có **[mở trên GitHub]** là bấm được, mở đúng tệp và đúng dòng trong repo. Đoạn cấu hình trong tài liệu được trích thẳng từ tệp thật, số dòng bên trái là số dòng trong tệp.

Phần 1. Kiến thức nền cả nhóm phải nắm

1.1 Đề bài yêu cầu gì

Đề 06 của môn yêu cầu: **thiết kế một nền tảng học trực tuyến cho 1.000 sinh viên**. "Thiết kế" ở đây không phải vẽ giao diện, mà là trả lời bằng số các câu hỏi của người làm hệ thống:

- 1.000 sinh viên thì **cùng lúc có bao nhiêu người** dùng? Cần bao nhiêu **băng thông** (đường truyền mạng), bao nhiêu **dung lượng lưu trữ**?
- Hệ thống gồm những phần nào, nối với nhau ra sao (**kiến trúc**)?
- Dựng thử được không, **đo** được không: chậm ở đâu, chịu được bao nhiêu người?
- Khi hỏng (máy chết, mất dữ liệu) thì **khôi phục** thế nào, mất bao lâu, mất bao nhiêu dữ liệu?

Phiếu đề chia thành 4 yêu cầu và 3 bài kiểm thử. Nhóm gọi bằng lời thường như sau:

Phần của đề	Nội dung	Ai lo chính
Yêu cầu 1: thiết kế	Thiết kế web, dữ liệu, video, đăng nhập; phân biệt **tổng tài khoản** với **số người dùng cùng lúc**	Đăng
Yêu cầu 2: tính tài nguyên	Tính băng thông, lưu trữ cho ít nhất 2 tình huống, ghi rõ mọi giả định	Đăng
Yêu cầu 3: triển khai	Dựng 1 web học liệu nhỏ có ít nhất 3 tài nguyên (video, PDF, bài nghe); thử 3 mức tải	Thành (web), Đăng (lab)
Yêu cầu 4: đo và đề xuất	Đo thời gian trả lời, số request mỗi giây, lỗi, CPU/RAM; chỉ ra điểm nghẽn; sao lưu; phân quyền	Vân Anh (đo), Đức (sao lưu, quyền)
Kiểm thử 1	Máy khách mở được trang và tải đúng học liệu	Vân Anh, Thành
Kiểm thử 2	Chạy 3 mức tải, ghi số liệu, điều kiện đo lặp lại được	Vân Anh
Kiểm thử 3	Dừng dịch vụ rồi khôi phục; kiểm tra học liệu và thời gian gián đoạn	Đức

Hồ sơ phải nộp (mục "Hồ sơ nộp chung" của phiếu đề): báo cáo nhóm 12–18 trang; sơ đồ và bảng tài nguyên/IP/quyền; cấu hình, script, mã nguồn; README hướng dẫn dựng lại; bảng kiểm thử có giờ chạy; dữ liệu đo gốc; video demo 5–7 phút; báo cáo cá nhân. Tất cả nằm trong thư mục ho-so-nop.

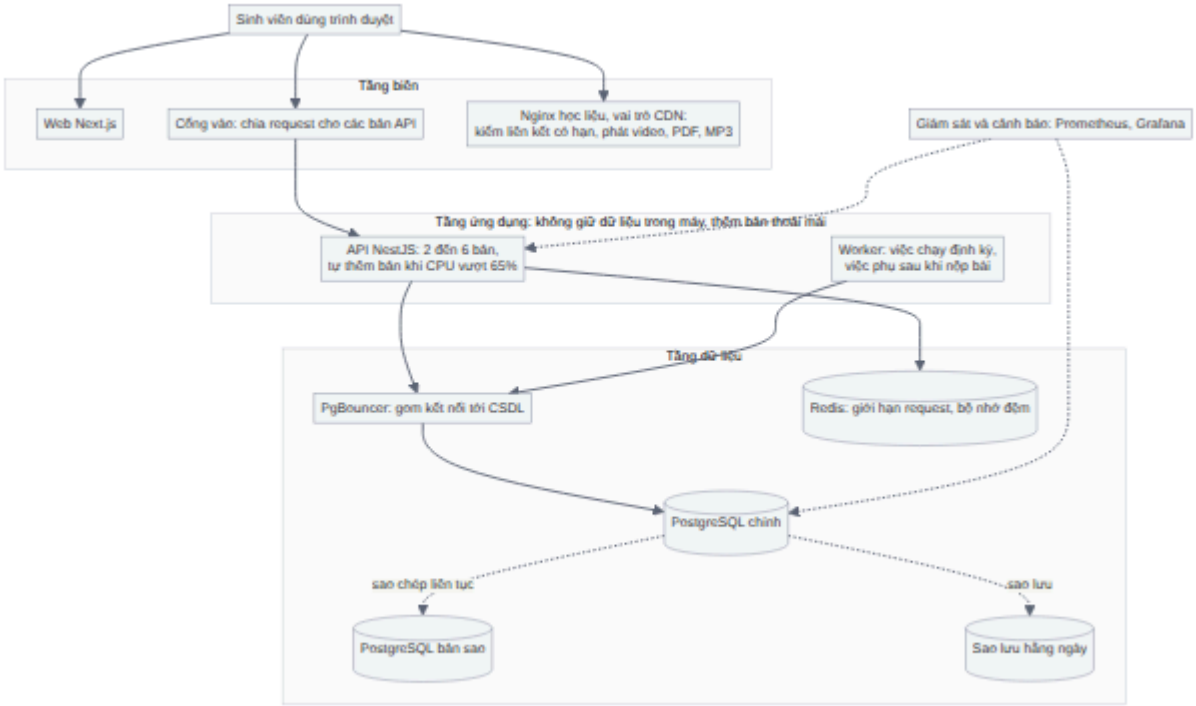
Buổi bảo vệ: mỗi người trình bày phần mình, **mở minh chứng** (tệp kết quả), **làm 1 thao tác hoặc sửa 1 thay đổi nhỏ** ngay trước mặt thầy, rồi **trả lời vấn đáp** (thầy có thể hỏi cả kiến trúc chung). Điểm cá nhân chiếm 50/100: 15 điểm phần kỹ thuật có bằng chứng, 15 điểm giải thích và thao tác trực tiếp, 20 điểm vấn đáp. Tài liệu này giúp bạn làm tốt cả 3 việc đó.

> Hanni chỉ là **hệ thống minh hoạ** (web học tiếng Trung mà Đăng viết từ trước). Thứ được chấm là cách nhóm thiết kế, đo, khôi phục, mở rộng, chứ không phải tính năng học tiếng Trung.

1.2 Hệ thống gồm những phần nào (ví như một quán ăn)

Hãy hình dung hệ thống như một quán ăn đông khách:

Phần trong hệ thống	Ví như	Làm gì
Trình duyệt của sinh viên	Khách hàng	Gửi yêu cầu, nhận kết quả, hiện lên màn hình
Web (Next.js, trang `/hoc-lieu`)	Thực đơn và bàn ăn	Phần giao diện sinh viên nhìn thấy, bấm vào
API (NestJS)	Bếp	Nhận yêu cầu ("cho tôi danh sách học liệu"), xử lý, trả kết quả. Chạy nhiều bản như nhiều đầu bếp
CSDL PostgreSQL	Kho nguyên liệu, sổ sách	Nơi giữ dữ liệu thật: tài khoản, tiến độ học, bài thi. Mất là mất thật
Redis	Bảng ghi chú dán tường	Ghi tạm những thứ cần đọc nhanh, mất cũng dựng lại được (bộ đếm giới hạn, bộ nhớ đệm)
PgBouncer	Người giữ cửa kho	Gom yêu cầu vào kho của nhiều đầu bếp, chỉ cho một số lượng cố định vào cùng lúc để kho không quá tải
Nginx học liệu (vai CDN)	Quầy phát đồ đóng gói sẵn	Phát thẳng video, PDF, MP3 cho khách, không bắt bếp phải làm. Khách phải có "phiếu" (liên kết có hạn)
Worker	Nhân viên hậu cần	Làm việc nền: nhắc học hằng ngày, xử lý việc phụ sau khi nộp bài
Giám sát (Prometheus, Grafana, Alertmanager)	Camera và chuông báo	Đo mọi thứ liên tục, vẽ biểu đồ, kêu khi có sự cố
Sao lưu	Bản photo sổ sách cất két	Mỗi ngày chép CSDL và học liệu ra chỗ khác để hỏng còn lấy lại được



Hình 1. Kiến trúc đề xuất cho 1.000 sinh viên (tệp gốc: ho-so-nop/so-do-va-bang/kien-truc-dich.png)

Ba tầng trong hình: tầng biên (web, cổng vào, Nginx học liệu) là chỗ khách đi vào; tầng ứng dụng (API, worker) là chỗ xử lý; tầng dữ liệu (PostgreSQL, Redis, sao lưu) là chỗ giữ dữ liệu. Ý quan trọng nhất: **tầng ứng dụng không giữ dữ liệu trong máy**, mọi thứ cần nhớ đều nằm ở CSDL hoặc Redis, nên muốn chịu nhiều người hơn chỉ cần **thêm bản API**, không phải sửa code.

1.3 Từ điển thuật ngữ (đọc kỹ, gặp lại ở khắp tài liệu)

Mạng và web

- **Request (yêu cầu)**: một lần trình duyệt hỏi máy chủ, ví dụ "cho tôi danh sách học liệu". Mỗi lần bấm nút có thể sinh ra vài request.
- **Response (trả lời)**: máy chủ đáp lại request, kèm một **mã trả về** (mã HTTP) cho biết kết quả.
- **API**: tập các "cửa" mà máy chủ mở ra để trình duyệt gọi vào, mỗi cửa là một **route** (đường dẫn), ví dụ `GET /api/resources` (lấy danh sách học liệu), `POST /api/resources/:id/link` (xin liên kết tải). `GET` là đọc, `POST` là gửi hoặc tạo mới.
- **Mã trả về HTTP** hay gặp trong dự án:

Mã	Nghĩa	Gặp ở đâu trong dự án
200	Thành công	Gọi API bình thường
401	Chưa đăng nhập	Khách xin liên kết video cần đăng nhập
403	Đã biết là ai nhưng không có quyền, hoặc chữ ký sai	Sửa chữ ký liên kết học liệu; người học gọi chức năng quản trị
404	Không có thứ này (hoặc giấu đi cho an toàn)	Gọi `/api/metrics` từ ngoài
410	Đã từng có nhưng hết hạn	Dùng liên kết học liệu đã quá 10 phút
429	Gửi quá nhiều, chờ chút rồi thử lại	Một tài khoản gọi quá 300 request trong 1 phút
503	Máy chủ đang quá tải hoặc chưa sẵn sàng, "thử lại sau"	Cơ chế từ chối khi quá tải của nhóm; API mất CSDL
5xx khác (500, 502…)	Lỗi máy chủ thật	Trong các lượt đo chính thức: 0 lỗi loại này

- **Băng thông**: lượng dữ liệu đi qua đường mạng mỗi giây, đơn vị **Mbps** (megabit mỗi giây). Chú ý: 8 bit = 1 byte, nên 8 Mbps = 1 MB mỗi giây. Video 540p của nhóm là 2 Mbps, tức mỗi giây xem tốn khoảng 0,25 MB.

Máy chủ

- **CPU, lõi (core)**: bộ xử lý. "0,25 lõi" nghĩa là được dùng một phần tư sức một lõi. Trong Kubernetes viết `250m` (250 phần nghìn lõi).
- **RAM**: bộ nhớ đang làm việc, viết `256Mi` (256 mebibyte, gần bằng 256 MB). Hết RAM thì máy phải dùng **swap** (lấy ổ đĩa làm RAM tạm), chậm hẳn đi; vì vậy lần đo nào máy bị swap thì nhóm bỏ, đo lại.
- **Tiến trình**: một chương trình đang chạy, ví dụ 1 bản API là 1 tiến trình Node.js.
- **Event loop (vòng xử lý của Node.js)**: Node.js xử lý mọi request bằng một vòng lặp duy nhất. Khi CPU quá bận, vòng này bị **trễ** (event loop lag): request phải xếp hàng chờ dù chưa cần tới CSDL.

Nhóm đo độ trễ này để biết API đang quá tải.

Docker và Kubernetes

- **Docker, container, image**: Docker đóng gói một chương trình cùng mọi thứ nó cần vào một **image** (như hộp cơm đóng sẵn). Chạy image lên thành **container** (hộp cơm đang được ăn). Nhờ vậy máy nào có Docker cũng chạy được giống hệt nhau. Mọi công cụ của lab (k6, ffmpeg, kubectl) đều chạy trong container nên máy không phải cài gì thêm.
- **Kubernetes**: phần mềm **quản lý nhiều container** trên nhiều máy: tự bật lại container chết, tự thêm bản khi đông, chia request đều. **k3s** là bản Kubernetes nhẹ, nhóm chạy nó ngay trong Docker để dựng lab trên 1 máy.
- **Cụm (cluster)**: tập các máy Kubernetes quản lý. **Node**: mỗi máy trong cụm. Lab có 2 node: `k3s-server` (172.25.0.2) và `k3s-agent` (172.25.0.3).
- **Pod**: đơn vị nhỏ nhất Kubernetes chạy, thường là 1 container. 1 bản API = 1 pod.
- **Deployment**: bản mô tả "tôi muốn chạy N pod giống nhau từ image X". Kubernetes giữ đúng N pod, chết cái nào tạo lại cái đó.
- **Bản (replica)**: mỗi pod giống nhau trong một Deployment. "API chạy 6 bản" là 6 pod API cùng phục vụ.
- **Service**: một địa chỉ cố định đứng trước nhiều pod, chia request cho các pod đó. Pod chết, đổi địa chỉ, Service vẫn giữ nguyên.
- **Ingress (cổng vào)**: cửa chính từ ngoài vào cụm, xem đường dẫn để đưa request tới đúng Service (`api.hanni.localhost` tới API, `media.hanni.localhost` tới Nginx học liệu). Lab dùng Traefik làm cổng vào, mở ở `127.0.0.1:18080`.
- **Namespace**: ngăn chia trong cụm. Ứng dụng nằm ở `hanni`, giám sát nằm ở `observability`.
- **Manifest (tệp YAML)**: tệp mô tả thứ muốn chạy (Deployment, Service…). Nhóm sửa manifest rồi **apply** (nạp) vào cụm.
- **kubectl**: lệnh điều khiển cụm. Trong lab gọi qua `scripts/kubectl` (đã nối sẵn vào lab). Vài lệnh hay dùng: `scripts/kubectl -n hanni get pods` (xem các pod), `get hpa` (xem tự mở rộng), `logs <tên pod>` (xem nhật ký).
- **HPA (tự mở rộng)**: Horizontal Pod Autoscaler, cứ 15 giây đo CPU trung bình các bản API; vượt ngưỡng (65% mức CPU yêu cầu) thì thêm bản, thấp lâu thì bớt, trong khoảng 2–6 bản.
- **Probe (kiểm tra sức khoẻ)**: Kubernetes định kỳ gọi thử vào pod. **Readiness** hỏi "sẵn sàng nhận khách chưa?", chưa thì tạm không chia request cho pod đó. **Liveness** hỏi "còn sống không?", nhiều lần không trả lời thì khởi động lại pod.

Dữ liệu

- **CSDL (cơ sở dữ liệu) PostgreSQL**: nơi lưu dữ liệu dạng bảng (bảng `User`, `ReviewLog`…), mỗi dòng là một bản ghi.
- **Kết nối CSDL, pool**: mỗi lần API nói chuyện với CSDL phải dùng một **kết nối**. Pool là "hồ" kết nối mở sẵn để dùng lại. CSDL chỉ chịu được một số kết nối nhất định; hết kết nối thì request phải **xếp hàng chờ**, đây là điểm nghẽn đầu tiên nhóm đo được.
- **Bản sao (replica) CSDL**: một CSDL thứ hai liên tục chép theo CSDL chính, dùng để đọc bớt và để dự phòng khi CSDL chính hỏng.
- **WAL (nhật ký thay đổi)**: PostgreSQL ghi mọi thay đổi vào nhật ký trước. Nhờ nhật ký này mà bản sao chép theo được, và có thể khôi phục về **đúng một thời điểm** trước khi xoá nhầm.

- **Redis**: kho dữ liệu cực nhanh nằm trong RAM, dùng cho thứ mất cũng không sao: bộ đếm giới hạn request, bộ nhớ đệm.
- **Bộ nhớ đệm (cache)**: lưu tạm kết quả đã tính để lần sau trả ngay, không phải tính lại. Nhóm lưu tạm "khung lộ trình học" 60 giây.

Đo hiệu năng

- **Tải**: lượng người và request đổ vào hệ thống. Đo bằng **request/giây** hoặc **lượt/giây** (1 lượt mở bài = 5 request đọc).
- **Người dùng cùng lúc (phiên đồng thời)**: số người đang thật sự dùng ở cùng một thời điểm. 1.000 tài khoản không có nghĩa 1.000 người cùng lúc: tự học rải rác chỉ khoảng **63 người**, cả lớp xem video 300, thi thử đồng loạt 600.
- **Định luật Little**: số người cùng lúc = số người bắt đầu mỗi phút × số phút mỗi người ở lại. Ví như quán ăn: mỗi phút có 2,5 khách vào, mỗi khách ngồi 25 phút, thì lúc nào cũng có khoảng 63 khách trong quán.
- **Thời gian trả lời** (độ trễ): từ lúc gửi request tới lúc nhận trả lời, đơn vị **ms** (mili giây, 1.000 ms = 1 giây).
- **p50, p95, p99 (phân vị)**: xếp thời gian trả lời của mọi request từ nhanh tới chậm. **p95 = 40 ms** nghĩa là 95% request nhanh hơn 40 ms, 5% chậm hơn. Nhóm dùng p95 vì **trung bình** bị số đông nhanh che mất những người phải chờ lâu.
- **Thông lượng**: số request trả lời được mỗi giây.
- **Tỉ lệ lỗi**: phần trăm request bị lỗi.
- **SLO (ngưỡng đạt)**: mức chất lượng nhóm tự đặt ra: **95% lượt đọc dưới 300 ms, 95% lượt ghi dưới 500 ms, lỗi dưới 0,5%**. Đo mà vượt ngưỡng là "không đạt".
- **Điểm gãy**: mức tải cao nhất còn đạt SLO; tăng quá đó là chậm vọt. Lab đo được khoảng 40 lượt/giây (khoảng 200 request/giây).
- **Điểm nghẽn (nút thắt)**: bộ phận bị đầy đầu tiên khiến cả hệ thống chậm. Như con đường: chỗ hẹp nhất quyết định xe chạy được bao nhiêu.
- **Từ chối khi quá tải**: khi quá tải, API trả lời ngay mã 503 "thử lại sau" cho một phần lượt đọc, thay vì nhận hết rồi bắt tất cả chờ 5 giây. Người được phục vụ thì nhanh, người bị từ chối thì biết để thử lại.
- **k6**: công cụ **giả lập nhiều người dùng** gửi request đều đặn rồi đo thời gian trả lời. Nhóm viết **kịch bản** k6 bằng JavaScript.
- **Lab**: môi trường đo riêng của nhóm (cụm k3s 2 node trong Docker, dữ liệu giả 1.000 tài khoản), tách hẳn web thật để đo không ảnh hưởng người dùng.
- **GitHub Actions**: dịch vụ của GitHub cho mượn máy ảo chạy lệnh tự động. Laptop của nhóm không đủ RAM nên **số đo chính thức** chạy trên máy ảo GitHub (4 nhân, 16 GB) ngày 09/11/2026; máy ảo dựng lại lab từ đầu theo README rồi mới đo.

Giám sát và sao lưu

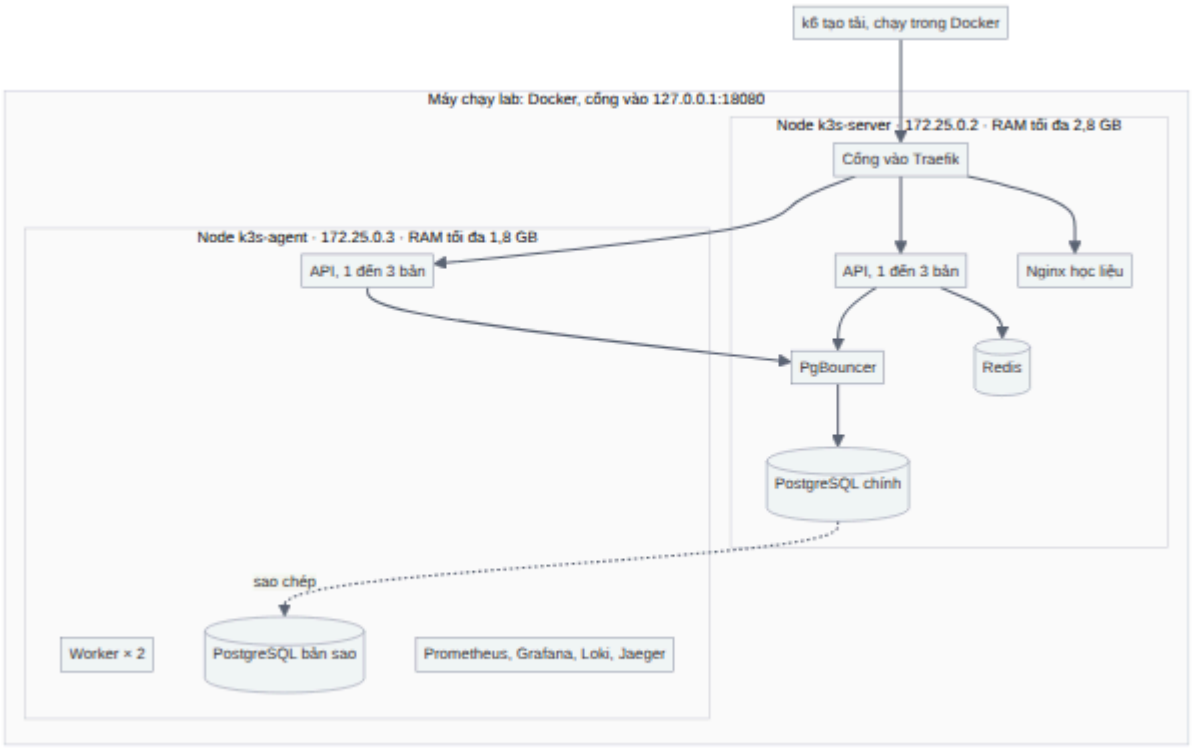
- **Prometheus**: cứ vài giây hỏi số liệu của từng thành phần (API, CSDL, Redis) rồi lưu lại theo thời gian.
- **Grafana**: vẽ số liệu của Prometheus thành biểu đồ. Nhóm có 3 bảng: chất lượng dịch vụ, mở rộng (số bản API, CPU), dữ liệu.

- **Alertmanager**: gửi cảnh báo khi một luật bị vi phạm, ví dụ "từ ngoài gọi API không được quá 30 giây".
- **Blackbox (dò từ ngoài)**: một chương trình đứng ngoài gọi thử API mỗi vài giây như một sinh viên thật, để biết sinh viên có vào được không.
- **Loki, Jaeger**: Loki gom nhật ký (log) của mọi pod; Jaeger lần theo đường đi của một request qua các phần.
- **Sao lưu (backup)**: chép dữ liệu ra một bản riêng để khi hỏng lấy lại được. **Khôi phục (restore)**: đưa bản sao lưu trở lại.
- **RPO**: lượng dữ liệu tối đa có thể mất, tính từ lần sao lưu gần nhất. Sao lưu mỗi ngày thì RPO tối đa 1 ngày.
- **RTO**: thời gian từ lúc hỏng tới lúc dùng lại được. Lab đo được RTO 52 giây khi xoá hẳn CSDL.
- **Mã băm SHA-256**: "dấu vân tay" của tệp, một chuỗi 64 ký tự tính từ nội dung. Sai một byte là chuỗi khác hẳn, nên so mã băm biết tệp tải về hay khôi phục ra có đúng y nguyên không.

Công cụ dựng lab

- **make, Makefile**: `make up` là chạy một nhóm lệnh đã viết sẵn trong tệp `Makefile`. Nhóm có `make up`, `make images`, `make deploy`, `make seed`, `make observability`, `make status`, `make stop`, `make start`, `make down`.
- **YAML**: định dạng các tệp cấu hình. Quy tắc: `khoá: giá trị`, **thụt lề bằng dấu cách** thể hiện cấp (sai thụt lề là hỏng tệp), dòng bắt đầu bằng `#` là ghi chú, chuỗi số nên để trong ngoặc kép như `"600"`.

1.4 Lab của nhóm trông như thế nào



Hình 2. Sơ đồ lab: 2 node, địa chỉ IP và các thành phần (tệp gốc: ho-so-nop/so-do-va-bang/so-do-lab.png)

Thành phần	Địa chỉ	Ghi chú
Node k3s-server	172.25.0.2	RAM tối đa 2,8 GB
Node k3s-agent	172.25.0.3	RAM tối đa 1,8 GB
Cổng vào (ingress)	127.0.0.1:18080	Mọi địa chỉ dưới đây đi qua cổng này
API	http://api.hanni.localhost:18080/api/health	
Grafana	http://grafana.hanni.localhost:18080	Tài khoản `admin`, mật khẩu lấy bằng lệnh ở README
Prometheus	http://prometheus.hanni.localhost:18080	
Alertmanager	http://alertmanager.hanni.localhost:18080	
Web	http://hanni.localhost:3000	Chạy riêng bằng `npx next dev`

1.5 Repo này có gì, phần của mình ở đâu

Đường dẫn	Nội dung
ho-so-nop/bao-cao/	Báo cáo nhóm (.docx, .pdf), 4 báo cáo cá nhân theo mẫu của thầy
ho-so-nop/so-do-va-bang/	2 sơ đồ, bảng tài nguyên/IP/quyền kèm bảng tính tài nguyên
ho-so-nop/cau-hinh-ma-nguon/hanni-server/infra/	**Toàn bộ cấu hình lab**: k8s/ (Kubernetes), k6/ (kịch bản tải), scripts/ (script kiểm thử, sao lưu), observability/ (giám sát), media/ (học liệu)
ho-so-nop/cau-hinh-ma-nguon/hanni-server/src/	Mã API nhóm viết cho đề: học liệu, số liệu đo, giới hạn request, từ chối khi quá tải
ho-so-nop/cau-hinh-ma-nguon/hanni-client/	Trang web học liệu và test tự động
ho-so-nop/data/raw/	Dữ liệu đo gốc, không sửa tay. `gha-09-11/` là lượt đo chính thức
ho-so-nop/evidence/	Minh chứng: ảnh chụp, nhật ký, danh sách mã băm. Danh mục ở `evidence/README.md`
ho-so-nop/bang-kiem-thu.xlsx	Bảng kiểm thử 62 dòng có giờ chạy
ho-so-nop/video-demo.mp4	Video demo 6 phút
Hanni-RIPT1304-Phan-cong.xlsx	Sheet phân công: việc của từng người, tiến độ, tab **Ôn vấn đáp**

Mã nguồn đầy đủ (chạy được) ở 2 repo: hanni-server nhánh feat/scaling và hanni-client nhánh feat/hoc-lieu. Thư mục `cau-hinh-ma-nguon/` chỉ là bản trích các tệp nhóm viết cho đề.

1.6 Dựng lab trên máy của bạn

Cần: Docker (kèm Docker Compose), `make`, `git`, còn trống khoảng 3 GB RAM và 8 GB đĩa. Windows thì dùng WSL 2.

```
git clone -b feat/scaling https://github.com/DANG-PH/hanni-server.git
cd hanni-server/infra
make up              # tạo học liệu mẫu, bật 2 node k3s, sinh mật khẩu ngẫu nhiên
make images          # build image API rồi nạp vào 2 node
make deploy          # chạy PostgreSQL + bản sao, PgBouncer, Redis, API, worker, Nginx học liệu, tự
mở rộng, chặn mạng
make seed            # tạo bảng, nạp từ vựng, tạo 1.000 tài khoản giả
make observability   # chạy Prometheus, Alertmanager, Grafana, Loki, Jaeger
make status          # mọi pod phải ở trạng thái Running
```

Lần đầu mất 10–20 phút vì phải tải image; các lần sau khoảng 4–5 phút. Tạm tắt cho nhẹ máy: `make stop`, bật lại: `make start`, xoá sạch: `make down`. Nếu máy yếu không dựng nổi, nhờ Đăng chạy trên GitHub Actions (đẩy nhánh lên `do-lab` là máy ảo tự dựng và đo).

1.7 Quy tắc chung khi sửa cấu hình

1. Mở tệp cần sửa trong `hanni-server/infra` (đường dẫn ở bảng "Sửa nhanh" của phần riêng bạn), sửa đúng dòng, giữ nguyên thụt lề.
2. Nạp lại vào cụm: `make deploy`.
3. Nếu đã sửa `k8s/app/config.yaml` hoặc `k8s/media-manifest.json` thì chạy thêm `scripts/kubectl -n hanni rollout restart deployment/hanni-api`, vì API chỉ đọc 2 tệp này lúc khởi động.
4. Nếu đã sửa luật cảnh báo trong `observability/` thì chạy `make observability` rồi `scripts/kubectl -n observability rollout restart deployment/prometheus`.
5. **Kiểm tra lại** bằng lệnh hoặc trang ghi ở cột "Kiểm tra" và nói được kết quả thay đổi thế nào.

> Khi thầy bảo "đổi X rồi chạy lại", hãy nói to 3 ý: (1) X nằm ở tệp nào, dòng nào; (2) đổi xong thì hệ thống sẽ thay đổi gì; (3) lệnh nào kiểm tra được điều đó. Làm như vậy là ăn trọn điểm "giải thích và thao tác trực tiếp".

Phần 2. Phần việc của bạn

2.1 Phiếu đề giao gì cho bạn

Phiếu đề ghi Sinh viên 2 lo **"Triển khai Web học liệu"**: dựng 1 web nhỏ có **ít nhất 3 tài nguyên** để sinh viên xem, tải. Đây là Yêu cầu 3 của đề.

Vai trò	Bạn làm gì	Phục vụ phần nào của đề
Frontend Developer	Trang `/hoc-lieu`: danh sách, xem video, tải PDF, nghe bài	Yêu cầu 3
UI/UX	Đủ 4 trạng thái, dùng tốt trên điện thoại 360 px, không nhảy bố cục	Yêu cầu 3
Media/CDN Engineer	Đóng gói video HLS 2 mức, mã băm, Nginx phát học liệu bằng liên kết có hạn	Yêu cầu 1–3 (nội dung video, băng thông)
Frontend QA & Web Performance	Test tự động, Lighthouse, HAR, so sánh HLS với MP4	Kiểm thử 1; số đo thay giả định ở Yêu cầu 2
Biên tập hồ sơ, video demo	Ghép báo cáo nhóm, video demo 5–7 phút	Hồ sơ nộp

> Vì sao frontend vẫn là phần của bài thiết kế hệ thống: thứ tốn tài nguyên nhất của nền tảng là **video**. Phần của bạn quyết định video đi qua đâu (Nginx/CDN, không qua API), mỗi người xem tốn bao nhiêu Mbps, và đưa ra số đo thật từ trình duyệt cho bảng tính.

2.2 Sản phẩm của bạn trong hồ sơ

Sản phẩm	Ở đâu
3 học liệu và script dựng	infra/media/, danh sách ở k8s/media-manifest.json
Danh sách mã băm	evidence/KT1/manifest.sha256
Trang web học liệu	app/(site)/hoc-lieu/page.tsx, components/resources/, lib/resources.ts
Test tự động	tests/hoc-lieu.test.ts
Nginx phát học liệu	k8s/app/media.yaml
Ảnh trang 360/390/1440 px	evidence/SV2/
Lighthouse, HAR	data/raw/lighthouse/, data/raw/har/
So sánh HLS với MP4	data/raw/gha-09-11/ket-qua-phu/so-sanh-video/
Báo cáo cá nhân	Lop_Nhom_06_MSSV_LeDinhThanh_BaoCaoCaNhan.docx
Video demo	video-demo.mp4

Phần 3. Học liệu: video, HLS, mã băm

3.1 Ba học liệu là gì

Mã	Học liệu	Định dạng	Ai được mở
R1	Bài giảng "Bốn thanh điệu tiếng Trung", 66 giây	Video HLS 2 mức 540p (2 Mbps) và 720p (3 Mbps), đoạn 6 giây	Phải đăng nhập
R2	Bảng phiên âm và 300 từ HSK 1	PDF 6 trang	Ai cũng tải được
R3	Gói nghe bài 1 "Chào hỏi", 10 từ	MP3 128 kbit/s, 27 giây	Ai cũng nghe được

Cả 3 do nhóm tự làm từ slide và ghi âm của Hanni (giấy phép mở), nên **không dính bản quyền**.

3.2 Video, bitrate, HLS: giải thích từ đầu

- **Bitrate**: số bit video dùng cho mỗi giây hình. 2 Mbps nghĩa là mỗi giây video tốn 2 triệu bit (0,25 MB). Bitrate càng cao hình càng nét nhưng càng tốn băng thông.
- **540p, 720p**: độ phân giải (chiều cao khung hình 540 hoặc 720 điểm ảnh).
- **MP4 tải thẳng**: một tệp duy nhất, trình duyệt tải dần từ đầu tới cuối. Đơn giản, lên hình nhanh, nhưng chỉ có 1 chất lượng: mạng chậm thì phải dừng chờ tải (giật).
- **HLS** (HTTP Live Streaming): video được **cắt thành nhiều đoạn nhỏ 6 giây** (tệp `.ts`), làm sẵn **nhiều mức chất lượng**. Có 1 tệp danh sách `.m3u8` ghi "có những mức nào, mỗi mức gồm những đoạn nào". Trình phát tải danh sách trước, rồi tải từng đoạn; **mạng chậm thì tự chuyển xuống mức thấp** để không giật. Mỗi đoạn là 1 tệp riêng nên CDN lưu tạm được từng đoạn.
- **hls.js**: thư viện JavaScript giúp Chrome, Android phát được HLS (Safari phát sẵn).

Cấu trúc thư mục video sau khi dựng (`infra/media/out/video-mau/`):

```
master.m3u8          ← danh sách chính: có 2 mức 540p và 720p
540p/index.m3u8      ← danh sách các đoạn của mức 540p
540p/seg-000.ts …    ← từng đoạn 6 giây (11 đoạn cho 66 giây)
720p/index.m3u8
720p/seg-000.ts …
bai-giang-720p.mp4   ← bản MP4 dùng để so sánh
```

Cách dựng: script make-sample.sh dùng **ffmpeg** (công cụ xử lý video, chạy trong Docker) ghép 6 slide với ghi âm thành video, rồi cắt HLS. Dòng quyết định bitrate:

hanni-server/infra/media/make-sample.sh, dòng 41–47 (mở trên GitHub):

```
  41 | # Chia đoạn 6 giây, giữ đúng bitrate trong bảng tính: 540p 2 Mbps, 720p 3 Mbps. Slide tĩnh
nén rất nhỏ nên phải ép CBR,
  42 | # nếu không số đo băng thông video không còn đúng với bảng tính.
  43 | for rung in 540p:960x540:2000k 720p:1280x720:3000k; do
  44 |   IFS=: read -r name size rate <<< "$rung"
  45 |   ffmpeg -i /out/.tam/bai-giang.mp4 -vf "scale=$size" \
  46 |     -c:v libx264 -preset veryfast -b:v "$rate" -minrate "$rate" -maxrate "$rate" -bufsize
"$rate" \
  47 |     -x264-params nal-hrd=cbr:force-cfr=1 -g 180 -keyint_min 180 -sc_threshold 0 -c:a aac -b:a
128k \
```

`-b:v`, `-minrate`, `-maxrate` cùng một giá trị nghĩa là **ép bitrate cố định** (CBR): slide tĩnh nén rất nhỏ, không ép thì video chỉ vài trăm kbit/s và phép đo băng thông không còn khớp bảng tính.

3.3 Mã băm SHA-256 và danh sách mã băm

Sau khi dựng, script tính **mã băm SHA-256** cho từng tệp (gồm cả từng đoạn video) và ghi vào `manifest.sha256`. Mã băm như dấu vân tay: sửa 1 byte là ra chuỗi khác hẳn. Kiểm thử 1 tải lại tất cả tệp như một máy khách rồi so mã băm: **30/30 tệp khớp** là chứng minh học liệu tới tay sinh viên đúng y nguyên.

3.4 Danh sách học liệu (manifest)

API biết có những học liệu nào nhờ tệp k8s/media-manifest.json. Mỗi mục:

hanni-server/infra/k8s/media-manifest.json, dòng 13–24 (mở trên GitHub):

```
13 |   {
14 |     "id": "bang-phien-am",
15 |     "title": "Bảng phiên âm và từ vựng HSK 1",
16 |     "description": "Tệp PDF 6 trang để in và học",
17 |     "kind": "pdf",
18 |     "dir": "bang-phien-am",
19 |     "file": "bang-phien-am.pdf",
20 |     "bytes": 419706,
21 |     "sha256": "3fff33657a056ec9baca7498de9187026c8a5e0222b0510844ee693b334e5caa",
22 |     "requiresLogin": false
23 |   },
24 |   {
```

Khoá	Ý nghĩa
`id`	Mã học liệu, dùng trong đường dẫn API `/resources/:id/link`
`title`, `description`	Chữ hiện trên thẻ ở trang
`kind`	`video`, `pdf` hoặc `audio`: quyết định nút "Xem", "Tải về" hay "Nghe"
`dir`, `file`	Thư mục và tên tệp trong `infra/media/out/`
`bytes`	Dung lượng, hiện trên thẻ
`sha256`	Mã băm (video để `null` vì gồm nhiều tệp, mã băm từng đoạn nằm trong manifest.sha256)
`requiresLogin`	`true`: khách phải đăng nhập mới mở được

Phần 4. Video đi đường nào: CDN, Nginx, liên kết có hạn

4.1 Đường đi khi sinh viên bấm "Xem video"

1. Trang `/hoc-lieu` gọi `GET /api/resources` lấy danh sách học liệu (từ manifest).
2. Sinh viên bấm "Xem video". Trang gọi `POST /api/resources/video-mau/link`.
3. API kiểm quyền: video cần đăng nhập, khách thì trả **401**, trang hiện "Đăng nhập để xem".
4. Đã đăng nhập thì API tạo **liên kết có chữ ký và thời hạn 10 phút**, dạng `http://media.hanni.localhost:18080/files/<chữ ký>/<thời điểm hết hạn>/video-mau/master.m3u8`.
5. Trình duyệt tải **thẳng từ Nginx**, không qua API. Nginx tự kiểm chữ ký và thời hạn.

> Vì sao làm vậy: cả lớp xem video cần gần 900 Mbps. Nếu video đi qua API thì API (Node.js) phải đẩy chừng ấy dữ liệu, sập trước khi kịp làm việc khác. Tách ra, API chỉ làm việc nhẹ là ký liên kết; phần nặng do Nginx (thực tế là **CDN**) gánh.

4.2 CDN là gì

CDN (mạng phân phối nội dung) là nhiều máy chủ đặt ở nhiều nơi, giữ sẵn bản sao các tệp tĩnh (video, ảnh, PDF) để người dùng tải từ chỗ gần nhất. Trong lab, **Nginx học liệu đóng vai CDN**. Khi lên thật chỉ cần thay địa chỉ `RESOURCE_BASE_URL` sang CDN, không phải sửa API hay trang web, vì trang chỉ dùng liên kết API trả về.

4.3 Nginx kiểm chữ ký như thế nào

hanni-server/infra/k8s/app/media.yaml, dòng 19–34 (mở trên GitHub):

```
19 |       location ~ ^/files/(?<tok>[^/]+)/(?<exp>[0-9]+)/(?<dir>[^/]+)/(?<file>.+)$ {
20 |         secure_link $tok,$exp;
21 |         secure_link_md5 "$secure_link_expires/$dir ${RESOURCE_LINK_SECRET}";
22 |         if ($secure_link = "") { return 403; }
23 |         if ($secure_link = "0") { return 410; }
24 |
25 |         alias /srv/hoc-lieu/$dir/$file;
26 |         types {
27 |           application/vnd.apple.mpegurl m3u8;
28 |           video/mp2t ts;
29 |           application/pdf pdf;
30 |           audio/mpeg mp3;
31 |         }
32 |         add_header Cache-Control "private, max-age=600";
33 |         add_header Access-Control-Allow-Origin "*";
34 |       }
```

Đọc từng dòng:

- `location ~ ^/files/(?<tok>…)/(?<exp>…)/(?<dir>…)/(?<file>…)` : tách liên kết thành 4 phần: chữ ký, thời hạn, thư mục, tên tệp.
- `secure_link $tok,$exp` : bật kiểm tra chữ ký `tok` và thời hạn `exp`.
- `secure_link_md5 "$secure_link_expires/$dir ${RESOURCE_LINK_SECRET}"` : Nginx tự tính lại MD5 của "thời hạn/thư mục khoá-bí-mật" theo **đúng công thức API đã dùng**. Khoá bí mật chỉ API và Nginx biết, nên người ngoài không tự tạo được liên kết.

- `if ($secure_link = "") return 403`: chữ ký sai (ai đó sửa liên kết) thì **403**.
- `if ($secure_link = "0") return 410`: chữ ký đúng nhưng đã quá hạn thì **410**.
- `alias /srv/hoc-lieu/$dir/$file`: tệp thật nằm ở thư mục này (gắn từ `infra/media/out` của máy vào node).
- `Cache-Control "private, max-age=600"`: trình duyệt được giữ tệp 10 phút, không giữ ở bộ đệm dùng chung.
- `Access-Control-Allow-Origin "*"` (CORS): cho phép trang web ở tên miền khác (hanni.localhost) tải tệp bằng JavaScript; hls.js cần điều này.

Ký theo thư mục chứ không theo tệp: danh sách `master.m3u8` trỏ tới các đoạn bằng đường dẫn tương đối (`540p/seg-000.ts`). Ký theo thư mục thì mọi đoạn trong `video-mau/` dùng chung 1 chữ ký, trình phát tải được hết mà không phải xin liên kết cho từng đoạn.

Phía API tạo chữ ký ở resources.service.ts (Đăng viết, xem tài liệu của Đăng mục 6.4).

Phần 5. Trang web học liệu

5.1 Các tệp của trang

Tệp	Làm gì
app/(site)/hoc-lieu/page.tsx	Trang chính: tiêu đề, danh sách thẻ, 4 trạng thái
components/resources/resource-card.tsx	1 thẻ học liệu: tên, loại, dung lượng, nút Xem/Tải/Nghe, xin liên kết khi bấm
components/resources/hls-video.tsx	Trình phát video dùng hls.js
lib/resources.ts	Hàm gọi API: `useResources()` lấy danh sách, `createResourceLink(id)` xin liên kết, `formatBytes()` đổi số byte ra KB/MB
lib/nav.ts	Thêm lối vào trang ở trang Luyện tập (không thêm mục menu)
tests/hoc-lieu.test.ts	5 ca test tự động trên trình duyệt thật

Đầu tệp trang có khối **BẢN ĐỒ TRANG** ghi trang gọi API nào, code server ở đâu:

hanni-client/app/(site)/hoc-lieu/page.tsx, dòng 3–15 (mở trên GitHub):

```
   3 | /**
   4 |  * BẢN ĐỒ TRANG — HỌC LIỆU (/hoc-lieu) — video, bài nghe, tài liệu tải về
   5 |  * Trang này gọi API nào (← file client gọi) và code server ở đâu (→ đường dẫn trong hanni-
server/src/). Sửa luồng gọi
   6 |  * API thì sửa khối này cho khớp.
   7 |  *
   8 |  *   GET /resources            ← useResources() ở lib/resources.ts
   9 |  *                               → server: modules/resources/resources.controller.ts →
ResourcesService.list()
  10 |  *   POST /resources/:id/link  ← createResourceLink() ở lib/resources.ts (←
components/resources/resource-card.tsx)
  11 |  *                               → server: modules/resources/resources.controller.ts →
ResourcesService.link()
  12 |  *
  13 |  * Khung quanh trang (menu, thanh trên, chuông, trợ lý AI) gọi API riêng — xem khối BẢN ĐỒ ở
app/(site)/layout.tsx.
  14 |  */
  15 |
```

5.2 Bốn trạng thái và vì sao cần

Trạng thái	Khi nào	Trang hiện gì
Đang tải	Đang chờ API trả danh sách	3 thẻ khung xám giữ chỗ (để chân trang không bị đẩy xuống khi dữ liệu tới)
Có dữ liệu	API trả danh sách	Các thẻ học liệu
Trống	Danh sách rỗng	Câu "chưa có học liệu" thay vì trang trắng
Lỗi	API lỗi, mạng rớt	Câu báo lỗi kèm nút **Thử lại**

Nhảy bố cục (CLS): khi dữ liệu tới làm các phần trên trang dịch chuyển, người dùng bấm nhầm. Lighthouse chấm bằng điểm CLS (càng nhỏ càng tốt, dưới 0,1 là tốt). Ban đầu trang có CLS **0,225**; thêm 3 thẻ khung giữ chỗ, còn **0,011**.

5.3 Đăng nhập và cookie

Đăng nhập xong, máy chủ gửi về **cookie** `hanni_access` (một mẩu dữ liệu trình duyệt tự gửi kèm mọi request sau đó để máy chủ biết bạn là ai). Trình duyệt chỉ gửi cookie cho đúng tên miền đã ghi trong cookie. Đã gặp lỗi thật: cookie ghi tên miền `localhost` trong khi API ở `api.hanni.localhost` nên trình duyệt bỏ cookie, đăng nhập xong vẫn là khách. Cách sửa: chạy web ở `hanni.localhost:3000` và đặt `COOKIE_DOMAIN=hanni.localhost` (trong config.yaml). Đây là "vấn đề đã gặp" trong báo cáo cá nhân của bạn.

5.4 Test tự động

Test mở trang trên Chrome thật (thư viện playwright-core), **giả lập API** để bật được từng trạng thái, rồi kiểm tra trang hiện đúng:

hanni-client/tests/hoc-lieu.test.ts, dòng 50–50 (mở trên GitHub):

```
50 | test("có danh sách: hiện đủ học liệu kèm loại và dung lượng", async () ⇒ {
```

5 ca: có danh sách; danh sách trống; API lỗi rồi bấm thử lại; khách gặp học liệu cần đăng nhập được mời đăng nhập; không xin được liên kết thì báo lỗi ngay trên thẻ. Chạy: `cd hanni-client && BASE_URL=http://hanni.localhost:3000 npm run test:hoc-lieu`. Kết quả chính thức: **5/5 đạt**.

Phần 6. Đo phía trình duyệt và Kiểm thử 1

6.1 Lighthouse

Lighthouse là công cụ có sẵn trong Chrome, chấm trang theo 4 nhóm (0–100): **Hiệu năng** (tải nhanh không), **Truy cập** (người khiếm thị, dùng bàn phím có dùng được không), **Thực hành tốt** (lỗi bảo mật, lỗi console), **SEO** (công cụ tìm kiếm đọc được không). Chỉ số con hay gặp: **LCP** (thời gian hiện nội dung lớn nhất), **CLS** (nhảy bố cục).

Chế độ	Hiệu năng (3 lần)	Truy cập	Thực hành tốt	SEO	CLS
Điện thoại (giả lập mạng chậm)	87, 89, 89	100	100	100	0,011
Máy tính	100, 100, 100	100	100	100	0,003

6.2 HAR: ghi lại mọi request của một phiên học

HAR là tệp Chrome xuất ra từ tab Network của DevTools (phím F12), ghi mọi request trang đã gửi, mỗi cái nặng bao nhiêu, mất bao lâu. Cách làm: F12 → Network → bật "Preserve log", "Disable cache" → học trọn bài 1 → chuột phải → "Save all as HAR". Nhớ xoá cookie trong tệp trước khi nộp. Kết quả: **20 request API, tổng 61 KB, trung bình 2 KB mỗi request**, cộng 10 tệp ghi âm 53 KB. Số này giúp Đăng thấy giả định 8 KB trong bảng tính là dư, an toàn.

6.3 So sánh HLS với MP4 (phần mở rộng tự chọn của bạn)

Cùng 1 video, cùng máy, cùng Chrome; mỗi lần xem 60 giây, tắt bộ nhớ đệm; 2 điều kiện mạng (thường và Slow 4G giả lập), mỗi kiểu 3 lần. Script: scripts/so-sanh-video.cjs.

Kiểu	Hình đầu tiên	Tải trong 60 giây	Dừng chờ tải	Xem được trong 60 giây
Mạng thường, HLS	362 ms	38,2 MB	0,0 giây	59,7 giây
Mạng thường, MP4	95 ms	37,6 MB	0,0 giây	59,9 giây
Mạng chậm, HLS	8.267 ms	11,7 MB	12,5 giây	39,3 giây
Mạng chậm, MP4	843 ms	12,0 MB	28,1 giây	31,1 giây

Đọc bảng: MP4 lên hình nhanh hơn vì không phải tải danh sách và thư viện phát. Nhưng mạng chậm thì HLS tự hạ xuống 540p nên **dừng chờ ít hơn một nửa** và xem được nhiều hơn. Kết luận theo số đo: dùng HLS cho bài giảng; MP4 chỉ hợp đoạn ngắn trên mạng tốt.

6.4 Kiểm thử 1: máy khách tải đúng học liệu

Script kt1.sh đóng vai trình duyệt: xin liên kết, tải đủ 30 tệp (danh sách và 22 đoạn video, PDF, MP3), so mã băm với `manifest.sha256`. Lượt chính thức chạy 3 lần: **3 lần, mỗi lần 30 tệp đúng, 0 tệp sai, 69174525 byte, 1.7 giây**.

Phần 7. Sửa và cấu hình trực tiếp

Bảng đầy đủ cho cả nhóm ở infra/README.md, mục "Sửa nhanh khi bảo vệ".

7.1 Thêm 1 học liệu (thao tác hay bị yêu cầu nhất)

1. Chép tệp vào thư mục mới, ví dụ `infra/media/out/de-cuong/de-cuong.pdf`.
2. Thêm 1 mục vào `infra/k8s/media-manifest.json` (nhớ dấu phẩy giữa các mục):

```
{
  "id": "de-cuong",
  "title": "Đề cương môn học",
  "description": "Tệp PDF 2 trang",
  "kind": "pdf",
  "dir": "de-cuong",
  "file": "de-cuong.pdf",
  "bytes": 120000,
  "sha256": null,
  "requiresLogin": false
}
```

1. `make deploy` (nạp lại danh sách), rồi `scripts/kubectl -n hanni rollout restart deployment/hanni-api` (API đọc danh sách lúc khởi động).
2. Mở http://hanni.localhost:3000/hoc-lieu: có thẻ mới, bấm "Tải về" mở được tệp.

7.2 Đổi học liệu thành cần đăng nhập (hoặc ngược lại)

Trong manifest, đổi `"requiresLogin": false` thành `true`, làm bước 3 như trên. Mở trang khi **chưa đăng nhập**: thẻ đó hiện "Đăng nhập để mở". Giải thích: API trả 401 cho khách, trang bắt mã đó để mời đăng nhập.

7.3 Đổi thời hạn liên kết

hanni-server/infra/k8s/app/config.yaml, dòng 35–35 (mở trên GitHub):

```
35 |   RESOURCE_LINK_TTL_SEC: "600"
```

Đổi `"600"` thành ví dụ `"30"`, `make deploy` + khởi động lại API. Xin liên kết, chờ 30 giây rồi mở lại liên kết đó: nhận **410**.

7.4 Đổi bitrate video

Sửa dòng `for rung in 540p:960x540:2000k 720p:1280x720:3000k` trong `media/make-sample.sh` (ví dụ 2000k → 1500k), xoá thư mục `media/out/video-mau`, chạy `make sample`. So dung lượng tệp trước và sau. Nhớ báo Đăng sửa ô bitrate trong bảng tính cho khớp.

7.5 Sửa chữ trên trang

Mở `hanni-client/app/(site)/hoc-lieu/page.tsx`, sửa chữ tiêu đề hoặc câu ở trạng thái trống. Trang đang chạy bằng `npx next dev` sẽ tự tải lại.

7.6 Cho thầy thấy lỗi 403

Xin 1 liên kết (F12 → Network, bấm "Tải về", xem phản hồi của `POST /resources/.../link`), sửa 1 ký tự trong phần chữ ký của liên kết rồi mở: Nginx trả **403**. Giải thích: chữ ký tính từ thời hạn, thư mục và khoá bí mật; sửa bất kỳ phần nào là không khớp.

Phần 8. Chạy lại và số liệu của bạn

8.1 Chạy lại phần của bạn, từng bước

1. Dựng lab (mục 1.6). Học liệu được tạo ở bước `make up` (lệnh `make sample` bên trong).
2. Chạy web: `cd hanni-client && npm install`, rồi `NEXT_PUBLIC_API_URL=http://api.hanni.localhost:18080/api npx next dev -H hanni.localhost -p 3000`.
3. Mở http://hanni.localhost:3000/hoc-lieu khi chưa đăng nhập: thấy 3 thẻ, video mời đăng nhập, PDF và MP3 mở được.
4. Đăng nhập tài khoản giả `loadtest-0001@example.test` (mật khẩu lấy bằng `scripts/kubectl -n hanni get secret hanni-secrets -o jsonpath='{.data.LOADTEST_PASSWORD}' | base64 -d`), xem video.
5. F12 → Network: thấy request `master.m3u8`, `index.m3u8`, các đoạn `.ts` tải từ `media.hanni.localhost`, không qua `api.`.
6. Chạy test: `BASE_URL=http://hanni.localhost:3000 npm run test:hoc-lieu` → 5 ca đạt.
7. Chạy Kiểm thử 1: `cd hanni-server/infra && scripts/kt1.sh k6/ket-qua/kt1.txt` → dòng cuối "30 tệp đúng, 0 tệp sai".

8.2 Số liệu để nói khi bảo vệ

Câu cần nói	Số
Kiểm thử 1	3 lần, mỗi lần 30/30 tệp khớp mã băm
Test tự động	5/5 ca đạt
Lighthouse điện thoại	Hiệu năng 87, 89, 89; truy cập, thực hành tốt, SEO 100; CLS 0,011 (trước khi sửa 0,225)
HAR 1 phiên học	20 request API, 61 KB, trung bình 2 KB/request
Mạng chậm, dừng chờ tải	HLS 12,5 giây, MP4 28,1 giây
Liên kết sai chữ ký / hết hạn	403 / 410

Phần 9. Câu hỏi vấn đáp

Trả lời ngắn, có số, rồi mở minh chứng ở cột cuối. Thầy hỏi khác đi một chút thì vẫn dựa vào ý chính của câu trả lời. Cùng nội dung có ở tab **Ôn vấn đáp** của sheet phân công.

Câu hỏi về phần của bạn

Vì sao chia video thành HLS?

Video cắt thành đoạn 6 giây, có 2 mức 540p và 720p. Trình phát chỉ tải phần đang xem và tự hạ chất lượng khi mạng chậm; CDN lưu tạm được từng đoạn.

Mở minh chứng: cau-hinh-ma-nguon/hanni-server/infra/media/make-sample.sh

Vì sao liên kết tải có hạn?

Để liên kết bị chia sẻ không dùng được mãi. API kiểm quyền rồi ký liên kết 10 phút; Nginx kiểm chữ ký: sai chữ ký trả 403, hết hạn trả 410.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt3/quyen.txt (dòng Q2, Q3)

HLS có luôn tốt hơn MP4 không?

Không. MP4 lên hình nhanh hơn (95 so với 362 ms ở mạng thường). Nhưng mạng chậm thì HLS dừng chờ 12,5 giây, MP4 28,1 giây. Nên dùng HLS cho bài giảng.

Mở minh chứng: Báo cáo nhóm mục 5.7

Số đo trình duyệt (HAR) giúp gì?

Thay giả định bằng số thật: 1 phiên học có 20 request API, trung bình 2 KB mỗi request, nhỏ hơn giả định 8 KB, nên băng thông API trong bảng tính là con số dư, an toàn.

Mở minh chứng: data/raw/har/; báo cáo nhóm mục 2.4

Trang chậm thì kiểm tra gì?

Mở DevTools, tab Network: request nào lâu, chờ máy chủ hay tải nội dung lâu, tệp nào nặng; sau đó chạy Lighthouse.

Mở minh chứng: data/raw/lighthouse/

Thao tác có thể bị yêu cầu

Thêm 1 học liệu vào k8s/media-manifest.json rồi make deploy + khởi động lại API, mở trang thấy thẻ mới; đổi requiresLogin; đổi thời hạn liên kết; phát video ở mạng chậm.

Mở minh chứng: cau-hinh-ma-nguon/hanni-server/infra/README.md, mục "Sửa nhanh khi bảo vệ"

Câu hỏi chung ai cũng phải trả lời được

Vì sao thử 20 kết nối không chứng minh phục vụ được 1.000 sinh viên?

Thời gian trả lời không tăng đều: từ 10 tới 40 lượt/giây p95 chỉ từ 23 lên 52 ms, thêm 10 lượt nữa đã 261 ms. Mức nhỏ chưa chạm giới hạn nào nên không nói gì về điểm gãy. Kết quả chỉ đúng cho đúng máy, dữ liệu, cấu hình đã đo.

Mở minh chứng: Báo cáo nhóm mục 5.3 (đo thăm dò), 6.2

Tài khoản khác người dùng cùng lúc thế nào?

Tài khoản là số người có thể dùng; người dùng cùng lúc mới tạo tải. 1.000 tài khoản sinh ra 63 người khi tự học, 300 khi cả lớp xem video, 600 khi thi thử đồng loạt.

Mở minh chứng: Báo cáo nhóm mục 2.1–2.2; so-do-va-bang/bang-tai-nguyen-ip-quyen.xlsx

Vì sao video không đi qua API?

Cả lớp xem video cần khoảng 888 Mbps trong khi API chỉ khoảng 56 request/giây. API chỉ kiểm quyền và ký liên kết, tệp do Nginx (thực tế là CDN) phát.

Mở minh chứng: Báo cáo nhóm mục 3.2; cau-hinh-ma-nguon/hanni-server/infra/k8s/app/media.yaml

Thêm máy chủ có hết quá tải không?

Không, nếu điểm nghẽn ở CSDL hoặc băng thông. Phải đo để biết điểm nghẽn trước: ở lab, nghẽn đầu tiên là hàng chờ kết nối CSDL, sửa xong mới tới CPU của API.

Mở minh chứng: Báo cáo nhóm mục 6.1; bảng Grafana "Mở rộng"

Bản sao lưu thế nào mới có giá trị?

Khi đã khôi phục thử, so số dòng và mã băm, đo được thời gian. Nhóm đã xoá hẳn CSDL rồi khôi phục: số dòng khớp, RTO 52 giây.

Mở minh chứng: Báo cáo nhóm mục 5.4; data/rpo-rto.xlsx

Lên 10.000 hay 100.000 sinh viên thì làm gì?

10.000: CSDL máy riêng có bản sao, kho tệp + CDN bắt buộc, khoảng 5 bản API. 100.000: nhiều vùng, tự chuyển CSDL khi hỏng, tách kho phân tích. Chuyển mức khi số đo chạm ngưỡng, không làm trước.

Mở minh chứng: Báo cáo nhóm mục 6.4

Phần 10. Việc cần làm trước khi nộp và bảo vệ

Trước khi nộp

1. Đọc báo cáo nhóm `ho-so-nop/bao-cao/Bao-cao-nhom.pdf` và báo cáo cá nhân của bạn `ho-so-nop/bao-cao/ca-nhan/Lop_Nhom_06_MSSV_LeDinhThanh_BaoCaoCaNhan.docx`.
2. Tự chạy lại phần của bạn theo mục "Chạy lại". Ghi lại bạn đã tự làm gì, rồi điền vào chỗ "Tự làm: ....." ở mục 7 và mục 9 của báo cáo cá nhân.
3. Điền mã sinh viên, lớp, nhóm vào báo cáo cá nhân; đổi tên tệp theo mẫu `Lop_Nhom_DeTai_MSSV_BaoCaoCaNhan.docx`.
4. **Chỉ ký mục 10 (cam kết) khi đã tự chạy lại và hiểu được.** Mục 9 đã ghi rõ nhóm dùng trợ lý AI soạn bản nháp; người ký chịu trách nhiệm giải thích.
5. Báo Đăng khi xong để gom hồ sơ nộp lên Drive của thầy.

Trước buổi bảo vệ

1. Tập **mở đúng minh chứng trong 10 giây**: biết tệp kết quả của mình nằm ở đâu trong `ho-so-nop/` (bảng "Sản phẩm của bạn" ở phần 2).
2. Tập **1–2 thao tác** ở phần "Sửa và cấu hình trực tiếp": nói được tệp nào, dòng nào, lệnh áp dụng, cách kiểm tra.
3. Tập trả lời các câu ở phần Câu hỏi vấn đáp bằng lời của mình, có số.
4. Xem lại phần 1 (kiến thức nền): thầy có thể hỏi kiến trúc chung, không chỉ phần của bạn.

> Câu chốt khi bị hỏi khó: "Theo số đo của nhóm trên lab, trong điều kiện ... thì ... ; giới hạn là ... ; muốn chắc hơn thì đo thêm ...". Trả lời theo số đo và nêu giới hạn luôn được đánh giá cao hơn khẳng định chung chung.